

Inconsistent Encoding and Its Consequences in Machine Learning

Dinis Afreixo Oliveira

Abstract—Encoding inconsistency is a pervasive yet often overlooked source of failure in machine learning pipelines. Unlike many data quality issues, it rarely surfaces as an explicit error — instead, it silently corrupts feature representations, degrades model performance, and can introduce bias in high-stakes applications. This paper characterizes encoding inconsistency across four principal forms: semantic, scheme-level, ordinal, and character-level. For each, the origins and downstream consequences are examined, including model performance degradation, fairness violations, reproducibility failures, and pipeline fragility. Detection and mitigation strategies are then reviewed and critically compared, spanning data profiling, schema validation, distribution monitoring, encoder serialization, and data contracts. The analysis concludes that while technical solutions are available and well-understood, the field’s tendency to prioritize encoding method selection over encoding consistency reflects an organizational gap rather than a scientific one — and one with measurable production consequences.

Index Terms—data quality, encoding inconsistency, categorical encoding, machine learning pipelines, train-serve skew, data contracts, algorithmic fairness, MLOps

I. INTRODUCTION

DATA quality is a critical concern in Machine Learning (ML) systems, commonly assessed by accuracy, completeness, consistency, and timeliness. In this work, we focus on consistency, where the same concept should be represented uniformly across a dataset.

Categorical data before learning have to be encoded into a structured numerical representation that algorithms can process. This process may introduce subtle errors, known as encoding inconsistencies. Inconsistent encoding occurs when same documented process/feature is represented through different formats, symbols or schemes across the dataset or pipeline.

ML algorithms rely on the assumption that the features presented are consistent, correctly labeled and that training and testing data follow the same distribution [1]. Encoding inconsistencies violate these assumptions without raising errors, the model silently learns with noise possibly predicting incorrect patterns. Inconsistencies introduce ambiguity in feature representation and can propagate across downstream systems, compromise model fairness when sensitive attributes are affected, and undermine reproducibility [2].

The real-world cost of these failures can be substantial: in 2022, Unity Technologies reported approximately USD 110 million in lost revenue attributed to underperforming ML models caused by corrupted data pipelines [3].

II. ENCODING METHODS

a) Label Encoding: Assigns a unique integer to each category (e.g., "Red" = 0, "Green" = 1). It is simple

and efficient but may introduce artificial ordering between categories.

b) One-Hot Encoding: Represents each category with a binary column indicating the presence of that category. This avoids ordinal relationships but increases dataset dimensionality.

Red	Green	Blue
1	0	0
0	1	0
0	0	1

c) Ordinal Encoding: Maps categories to integers according to a meaningful order (e.g., "Low" = 0, "Medium" = 1, "High" = 2).

d) Target Encoding: Encodes each category using the mean value of the target variable for that category. This approach can capture useful relationships between features and the target but may introduce data leakage if not applied carefully.

e) Frequency Encoding: Replaces each category with its frequency (or proportion) in the dataset. This method preserves information about how often categories appear while keeping the dimensionality low.

III. TYPES AND SOURCES OF ENCODING INCONSISTENCIES

A. Semantic Inconsistencies

Semantic inconsistencies occur when the same value is represented by different symbols or strings within the same feature. For example, boolean attributes may appear as "Yes", "True", or "1", or gender classification may be represented like "M", "MALE" or "Male".

These inconsistencies often originate from multiple data sources, manual data entry, or integration with datasets from different conventions, and no constraint validation is performed.

With this a model interprets "M" and "MALE", as different categories. This will artificially fragment feature representation, making the model receive contradictory training signals messing with the decision boundary.

For mitigation of the problem input validation and string normalization are critical during preprocessing data. To help detect this problem frequency analysis may show rare variants of common values.

B. Categorical Encoding Scheme Inconsistencies

This type of inconsistency occurs when the same feature is encoded via different schemes in different pipeline stages, such as training and testing. A case where this appears can be using

at inference one encoder like One-Hot, and at training label encoding, presenting to the algorithm a new column structure.

The primary source is pipeline fragmentation, with independent implementation of training and testing. This is dangerous because no exception is raised, while the model degrades performance.

Target encoding is a particularly high-risk variant. Because encoded values are derived from the statistical relationship between a category and the target variable, any inconsistency in how this computation is performed, computing target means on the full training set rather than within cross-validation folds. The consequence is not just a representational mismatch but a corruption of the statistical signal the model relies on, producing optimistic training metrics that fail to generalize.

Therefore, validation of feature distribution and encoding. Mitigation can be achieved by serializing encoders with the model, enforcing use in inference.

C. Ordinal Inconsistencies

Ordinal inconsistency arises when the assigned order to categorical values either varies across data sources, is imposed on features with no natural order, or contradicts real-world order.

A common example is for nominal features like "Red"→0, "Blue"→1, and "Green"→2, where no magnitude relationship exists. Another example is when two datasets are merged using different conventions, e.g., 0=BSc, 1=MSc, 2=PhD and 0=PhD, 1=MSc, 2=BSc, which reverses the ordinal meaning for part of the data.

Linear models and neural networks interpret these integers as continuous values, so inconsistent ordering introduces misleading patterns. Merged datasets without proper normalization create opposing signals, reducing predictive reliability. When natural ordering is ambiguous, One-Hot Encoding should be used.

D. Character-Level Inconsistencies

Character-level inconsistencies arise when the same text is stored or transferred using different character encoding standards. This changes the underlying byte representation of strings and can corrupt values even when they still appear syntactically valid.

A common example involves accented characters. The city name "São Paulo" stored in ISO-8859-1 but interpreted as UTF-8 may appear as "SÃ£o Paulo". In tabular datasets this can cause silent join failures, since the corrupted and original strings differ at the byte level.

These issues often occur when integrating systems with different encoding defaults. Legacy databases frequently use ASCII or ISO-8859-1, while modern software typically relies on UTF-8. CSV exports from spreadsheet tools are another frequent source because their encoding depends on local configuration.

In machine learning pipelines, corrupted text may be processed as valid input. In NLP tasks this introduces noise into tokenization and embeddings, while in tabular pipelines it can

fragment records so that the same real-world entity appears as multiple entries.

Detection generally involves identifying the encoding during ingestion with tools such as `chardet` or `charset-normalizer`, combined with Unicode normalization. In practice, enforcing UTF-8 across the entire pipeline is the most reliable mitigation.

TABLE I
TYPES AND SOURCES OF ENCODING INCONSISTENCIES

Type	Example	Primary Source	ML Impact
Semantic	"Yes" vs "1" vs "TRUE"	Multi-source data, absent input validation	Artificial category splitting, noisy decision boundary
Encoding Scheme	One-Hot at training vs Label Encoding at inference; target encoding leakage	Pipeline fragmentation, no encoder serialization	Train/serve skew, silent prediction degradation
Ordinal	Label Encoding applied to nominal features; reversed ordinal scales across datasets	Default encoding without domain validation	Spurious magnitude relationships in linear models
Character-Level	UTF-8 vs ISO-8859-1 producing mojibake	Legacy systems, locale-dependent CSV exports	Tokenization failure, silent join fragmentation

IV. CONSEQUENCES IN ML PIPELINES

A. Model Performance Degradation

Encoding inconsistencies artificially inflate feature cardinality and introduce contradictory training signals, as same real-world scenarios receive opposing labels. Model performance and decision boundary are silently degraded, being a difficult problem to detect in production.

The severity varies by model architecture:

- **Tree-based models:** are somewhat resilient to false ordinality since splitting is based on thresholds, but are still affected by semantic inconsistencies.
- **Linear models and neural networks:** are fully affected, since they interpret ordinal integer as continuous magnitudes, so learn process is a result of the quality of encoding.
- **Train/serve skew:** refers to a discrepancy between the data or preprocessing logic used during model training and the data encountered during real-time inference. It becomes a "silent killer" in production.

B. Fairness and Bias Amplification

Inconsistent encoding of sensitive attributes can mislead models into learning patterns tied to data entry rather than reality. For instance, recording gender as "F" in one source and "Female" in another creates artificial subgroups within the same demographic. A model trained on such data may

apply different decision thresholds to effectively the same individuals, purely based on how their data was recorded.

This problem is especially concerning in high-stakes areas like hiring, credit scoring, or medical diagnosis. The bias arises from technical inconsistencies, not explicit discrimination, making it harder to detect and audit. While most fairness research targets biased labels or sampling, encoding-induced bias is relatively understudied, highlighting a significant gap in current work on algorithmic fairness [6].

C. Reproducibility Failures

Reproducibility in ML requires that the same data processed through the same pipeline yields the same model. Encoding inconsistencies violate this: if encoding logic is not versioned, repeated runs can assign different integer codes or reorder One-Hot columns; if new data is added, frequency- or target-based encoders change mappings for existing categories. This is critical in regulated domains—healthcare, finance, or legal—where models must be auditable. Encoding drift between runs undermines traceability and reproducibility [2].

D. Pipeline Fragility and MLOps

In production ML, data moves through ingestion, validation, feature engineering, training, and testing, often handled by separate teams. Encoding choices made early become implicit contracts for downstream processes. If these contracts are not documented or enforced, a small change in the data schema can silently break a model in production.

This issue motivates the use of data contracts [4], which explicitly define expectations between data producers and consumers, including how features are encoded. Without them, encoding mismatches add brittleness, raise maintenance costs, and generate technical debt, as illustrated by the Unity Technologies example, where unnoticed data changes caused real system failures.

V. DETECTION AND MITIGATION STRATEGIES

Identifying and correcting encoding inconsistencies requires intervention at different stages of the pipeline. The approaches below are complementary — each covers blind spots the others cannot — yet most pipelines implement at most one.

A. Detection

1) *Data Profiling*: Tools such as `ydata-profiling` expose value counts and rare variants per feature. A semantic inconsistency surfaces as a low-frequency duplicate of a dominant value, flagging it for review. The limitation is that profiling is retrospective — problems are only found after they have entered the pipeline, and at scale the volume of flags can exceed practical review capacity.

2) *Schema Validation*: Frameworks like `Great Expectations` enforce explicit constraints at ingestion [5] — permissible value sets, expected types, allowed distributions. Unlike profiling, this operates proactively. The weakness is coverage: validation only catches violations of rules defined in advance. Novel variants from unanticipated sources pass silently unless explicitly excluded.

3) *Distribution Monitoring*: Once deployed, tools such as `Evidently AI` track statistical distance between training and serving distributions using metrics like KL divergence or Population Stability Index. The signal is inherently ambiguous — a distribution shift may reflect an encoding change, a genuine population change, or a collection artifact. Inconsistencies present from the start also remain invisible, as there is no clean baseline to detect them against.

B. Mitigation

1) *Encoder Serialization*: Fitting encoders once and reusing the persisted object at inference directly eliminates train/serve skew. Wrapping preprocessing and model steps in a `scikit-learn Pipeline` and storing it with `joblib` ensures encoding behavior is identical across runs. The residual risk is unseen categories — the default `handle_unknown='ignore'` silently zeroes them out rather than raising an alert, trading a hard failure for a silent one.

2) *Canonical Standardization*: Mapping all value variants to a single representation at ingestion prevents semantic fragmentation from propagating downstream. Enforcing UTF-8 with Unicode NFC normalization handles character-level corruption at the same stage. The ongoing cost is maintenance: dictionaries require manual curation and offer no coverage for variants not yet observed.

3) *Data Contracts*: The approaches above treat inconsistency after it has been introduced. Data contracts — formal schema agreements between producers and consumers, expressed as versioned testable specifications in frameworks like `Soda Core` [?] — prevent it from arising. The obstacle is not technical. Contracts impose process constraints on producers with no direct stake in ML outcomes, making adoption a governance problem that tooling alone cannot solve.

C. Discussion

Technical mitigations reduce damage; contracts address the source. The field’s preference for the former reflects implementation cost rather than effectiveness. Regardless of approach, a shared constraint remains: every mechanism here requires prior definition of what correct looks like. For inconsistencies introduced by unanticipated sources, no automated system reliably helps — domain expertise is the foundation these tools build on, not a fallback when they fail.

VI. CRITICAL ANALYSIS

The literature on data quality in ML dedicates considerable attention to encoding method selection — debating the merits of one-hot versus target encoding, ordinal versus label, frequency versus binary [9]. This framing is useful but incomplete. A pipeline that applies the wrong encoding consistently will degrade predictably and visibly. One that applies the right encoding inconsistently will degrade silently, in ways that are difficult to attribute and harder to reproduce.

This distinction matters because it changes where the real risk sits. The consequences examined in Section IV — performance degradation, fairness violations, reproducibility failures

— share a common property: none of them surface as errors. The model trains, the pipeline executes, predictions are served. The damage is absorbed into the model’s learned parameters and expressed only as reduced accuracy or skewed outputs, typically discovered late and traced back with difficulty. In this sense, encoding inconsistency is less like a bug and more like a slow structural fault — it compounds quietly until something downstream forces attention to it.

The mitigation landscape, reviewed in Section V, reinforces this reading. Effective solutions exist at both the technical and organizational level. Encoder serialization, schema validation, and canonical standardization are well-understood and widely available. Data contracts are conceptually straightforward. What is missing is not capability but prioritization — inconsistency prevention rarely receives dedicated investment until after a production failure has made the cost visible [?].

This points to a gap that is organizational rather than scientific. The encoding method debate produces papers, benchmarks, and library features. Encoding consistency produces operational checklists that few teams enforce. Closing this gap requires treating consistency as a first-class concern in ML pipeline design — not a preprocessing detail to be handled informally, but a property to be specified, tested, and monitored with the same rigor applied to model performance.

VII. CONCLUSION

Encoding inconsistency is a persistent and underappreciated source of failure in ML pipelines. Across its four principal forms — semantic, scheme-level, ordinal, and character-level — the defining characteristic is that damage propagates silently, without errors that would prompt investigation.

The detection and mitigation tools reviewed here are adequate for the problem. The gap is not technical. Inconsistency prevention remains an informal concern in most pipelines, addressed reactively rather than by design. Until consistency is treated with the same rigor as model performance, the consequences documented in this work will continue to appear in production systems — quietly, and often too late.

ACKNOWLEDGEMENT OF AI TOOLS

The following AI tools were used in the preparation of this work.

Claude (Anthropic) was used throughout the writing process to support structural planning, section drafting, and critical argumentation. It was selected for its ability to maintain context across a long document-building session and produce academic-register prose with minimal post-editing.

ChatGPT (OpenAI) was used for targeted clarification of specific technical concepts and cross-checking of definitions. It was selected for its accessibility and broad technical knowledge base.

Consensus.ai was used to identify peer-reviewed literature relevant to the topics covered. It was selected specifically because it retrieves results directly from academic sources and surfaces citation context alongside paper summaries, reducing the risk of relying on unverified claims.

All AI-generated content was reviewed, adjusted, and validated by the author. Final arguments, critical positions, and editorial judgement remain the author’s own.

REFERENCES

- [1] D. Sculley *et al.*, “Hidden technical debt in machine learning systems,” in *Proc. Advances in Neural Information Processing Systems (NeurIPS)*, vol. 28, 2015, pp. 2503–2511.
- [2] S. Amershi *et al.*, “Software engineering for machine learning: A case study,” in *Proc. IEEE/ACM Int. Conf. Software Engineering (ICSE)*, 2019, pp. 291–300.
- [3] T. Krantz and A. Jonker, *A Compounding Threat: The True Cost of Poor Data Quality*. IBM Think, Jan. 23, 2026. [Online]. Available: <https://www.ibm.com/think/insights/cost-of-poor-data-quality>
- [4] E. Breck, S. Cai, E. Nielsen, M. Salib, and D. Sculley, “The ML test score: A rubric for ML production readiness and technical debt reduction,” in *Proc. IEEE Int. Conf. Big Data*, 2017, pp. 1123–1132.
- [5] N. Polyzotis, M. Zinkevich, S. Roy, E. Breck, and S. Whang, “Data validation for machine learning,” in *Proc. 2nd SysML Conf.*, 2018.
- [6] N. Sambasivan *et al.*, “Everyone wants to do the model work, not the data work: Data cascades in high-stakes AI,” in *Proc. ACM CHI Conf. Human Factors in Computing Systems*, 2021, pp. 1–15.
- [7] T. Gebru *et al.*, “Datasheets for datasets,” *Commun. ACM*, vol. 64, no. 12, pp. 86–92, 2021.
- [8] K. Peng, T. Bhatt, and R. Dobbe, “Mitigating dataset harms requires stewardship: Lessons from 1000 papers,” in *Proc. NeurIPS Datasets and Benchmarks Track*, 2021.
- [9] E. Rahm and H. H. Do, “Data cleaning: Problems and current approaches,” *IEEE Data Eng. Bull.*, vol. 23, no. 4, pp. 3–13, 2000.